

RSAT: Decentralized Security Auditing in Non-Standard Environments

Technical Design and Implementation of Client-Side Forensic Automation

Date: December 2025

Repository: https://github.com/Kajahkura/RSAT_Project

Abstract

This document details the architectural design and implementation logic of the Remote Security Audit Tool (RSAT). It addresses the technical imperative of executing "one-shot" security assessments in high-latency, decentralized environments. By leveraging a zero-dependency Python architecture that interfaces directly with native OS kernels, the tool eliminates deployment friction while maintaining forensic accuracy. This report covers the privilege escalation strategies, cross-platform abstraction layers, and automated build pipelines required to deliver a professional-grade audit artifact.

1. Executive Summary

The traditional network perimeter has dissolved. The rapid shift toward remote work and Bring Your Own Device (BYOD) policies has decentralized the attack surface, rendering legacy network-based scanning tools insufficient. In this new landscape, the endpoint is the new perimeter.

This document outlines the engineering and strategic principles behind the Remote Security Audit Tool (RSAT), a bespoke, cross-platform artifact designed to execute "one shot" security assessments on Microsoft Windows and macOS. By leveraging a "zero dependency" architecture, RSAT enables security consultants to rapidly assess client security postures without the economic and technical friction of heavy agent-based software.

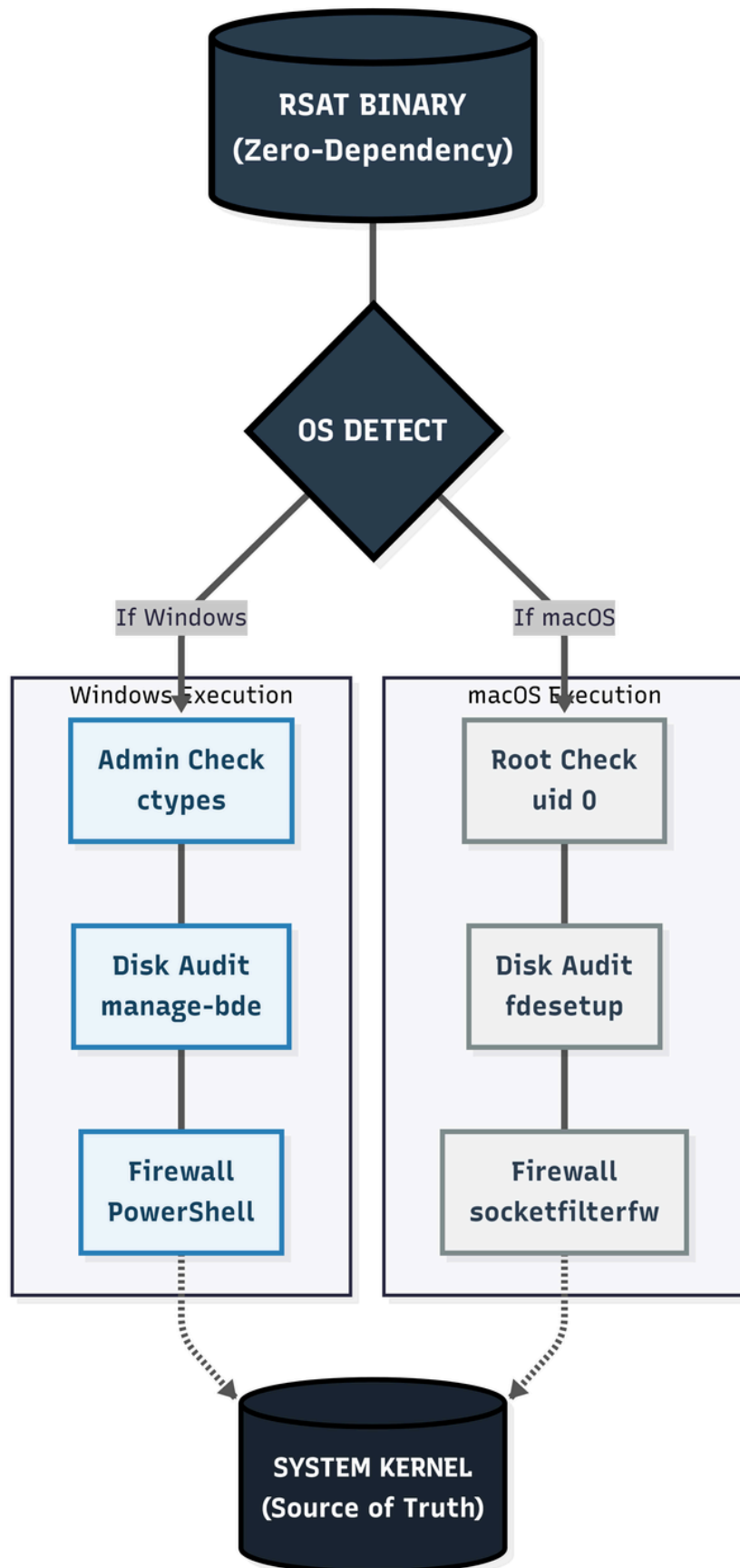
2. Strategic Motivation

For security consultancies operating in high volume markets, efficiency is an economic necessity. Traditional auditing methods often require manual intervention, complex installations, or reliance on the client's existing runtime environment—variables that introduce delay and error.

The RSAT was engineered to solve three specific market frictions:

- **Deployment Latency:** Eliminating the need for installation allows for immediate value delivery.
- **Environment Unpredictability:** Removing dependencies on client-side libraries prevents compatibility failures.
- **The "Source of Truth" Gap:** Direct querying of the OS kernel provides definitive data, superior to surface-level scans.

Section 2.2 (Cross-Platform Divergence)



3. Architectural Philosophy: The Hybrid Execution Model

The core innovation of RSAT is its "Hybrid Execution" architecture. While the logic and control flow are written in Python 3.10+ for its robust standard library, the heavy lifting is outsourced to native operating system binaries and system calls.

3.1 Zero-Dependency Distribution

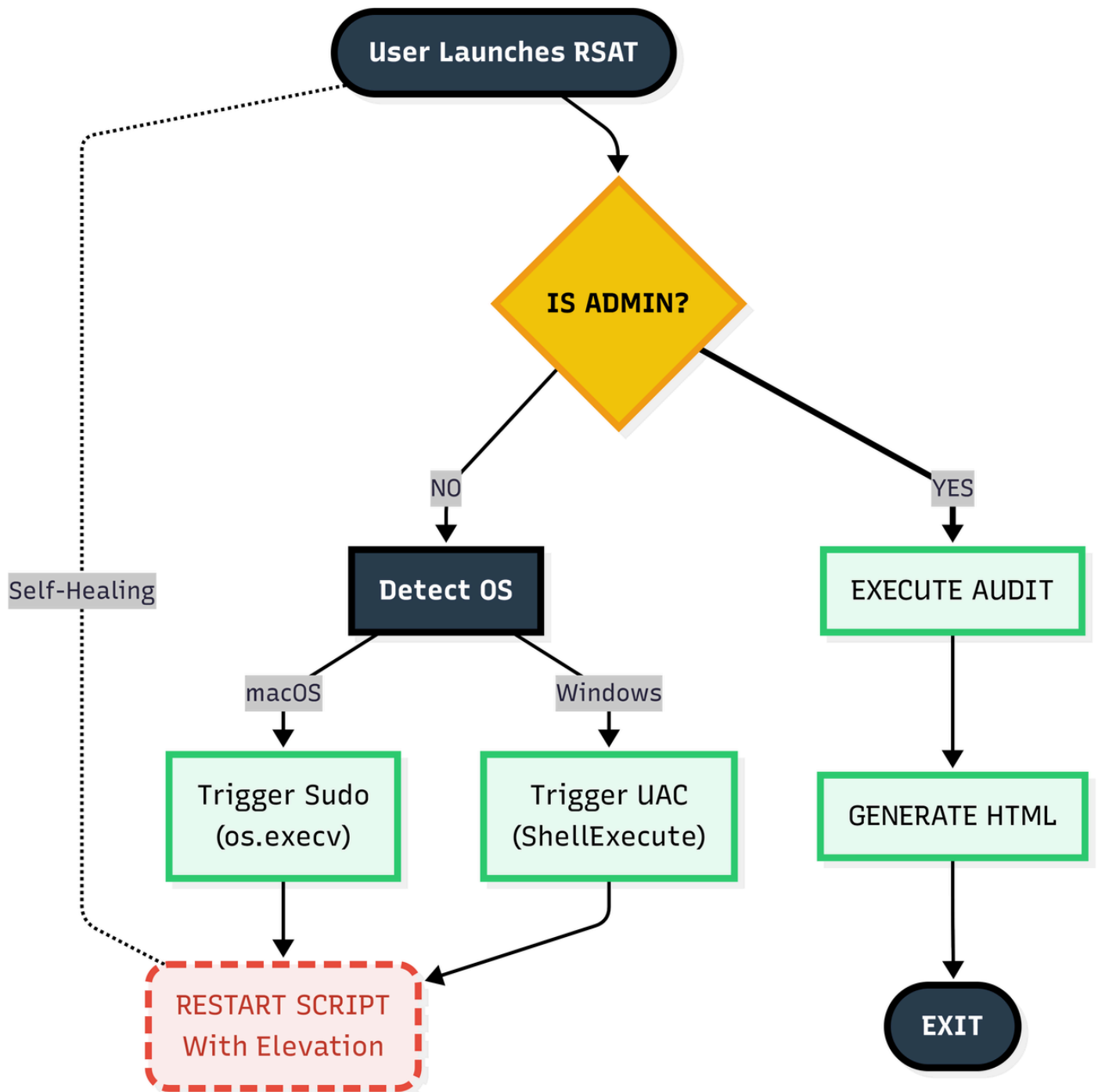
A critical vulnerability in many security tools is the assumption that the client environment is properly configured. RSAT eliminates this risk by utilizing **PyInstaller** to bundle the Python interpreter, scripts, and necessary dynamic link libraries (DLLs) into a single, standalone executable. This "frozen" binary approach ensures that the tool runs autonomously, requiring no pre-existing runtime environment on the client machine.

3.2 Operating System Abstraction Layer

To function seamlessly across divergent platforms, the tool implements a polymorphic "Operating System Abstraction Layer". Upon initialization, the system detects the host environment (Windows or macOS) and bifurcates the control flow.

- **Windows Implementation:** Leverages [ctypes](#) for low-level API calls and PowerShell for structured data queries.
- **macOS Implementation:** Utilizes POSIX-compliant system calls and native binaries like [fdsetup](#) and [socketfilterfw](#).

The Autonomous Privilege Escalation Loop



4. Deep Engineering Implementation

The value of RSAT lies in its ability to autonomously manage privileged operations and interrogate deep system controls.

4.1 Autonomous Privilege Management

Security auditing is an inherently privileged activity. Accessing firewall states or disk encryption keys requires Administrative (Windows) or Root (macOS) access.

- **Windows Escalation:** The tool uses [ctypes.windll.shell32.IsUserAnAdmin\(\)](#) to verify privileges. If insufficient, it leverages the [ShellExecuteW](#) API with the "runas" verb to trigger the User Account Control (UAC) prompt, allowing the script to restart itself with elevated rights while preserving command-line arguments.
- **macOS Escalation:** The tool checks for [UID 0](#). If the user is not root, it utilizes the [os.execv](#) strategy to replace the current process image with a [sudo](#) command, seamlessly prompting the user for authentication within the terminal.

4.2 Cryptographic & Perimeter Verification

The tool moves beyond simple registry checks to verify the active state of security controls.

- **Disk Encryption:** On Windows, it prioritizes the PowerShell [Get-BitLockerVolume](#) cmdlet for structured reliability but includes a robust regex parser for [manage-bde.exe](#) as a fallback for legacy systems. On macOS, it queries the CoreStorage or APFS Volume Manager via [fdesetup](#) to distinguish between "Encrypted," "Decrypted," and the critical edge case of "Encryption in Progress".
- **Network Defense:** The tool audits the Windows Defender Firewall "Profiles" (Domain, Private, Public) to ensure no dangerous mismatches exist. Simultaneously, a threaded port scanner checks the "Dirty Dozen" high-risk ports (e.g., SMB/445, Telnet/23) using concurrent sockets to identify active listening services on [localhost](#).

4.3 Forensics-Based Patch Auditing

Rather than triggering slow, bandwidth-intensive update scans, RSAT employs a forensic approach to determine patch latency.

- **Windows Registry Forensics:** It inspects specific keys in [HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing and WindowsUpdate\Auto Update](#) to detect "Pending Reboot" flags, which serve as a high-fidelity proxy for incomplete patch cycles.
- **macOS Plist Analysis:** It reads the cached [com.apple.SoftwareUpdate.plist](#) to instantly identify if the system has already acknowledged missing updates, reducing audit time from minutes to milliseconds.

5. Deployment & Continuous Integration

To professionalize the delivery pipeline, the project utilizes GitHub Actions to automate cross-platform compilation.

- **The Matrix Strategy:** The build pipeline provisions parallel Windows and macOS virtual machines to compile native binaries for each OS simultaneously.
- **Artifact Integrity:** This automated workflow ensures that every code push results in a fresh, reproducible "Release" containing [AuditTool_Windows.exe](#) and [AuditTool_macOS](#), solving the "cross-compilation myth" where developers mistakenly attempt to build Windows executables on Mac hardware.

6. Strategic FAQ for Leadership

Q: How does this tool handle modern privacy layers like macOS TCC (Transparency, Consent, and Control)?

A: The tool is engineered to handle *PermissionError* exceptions gracefully. When TCC blocks access to specific subsystems (despite root privileges), the reporting engine flags these as "Warnings" rather than "Failures." This prevents false positives that could alarm clients while accurately reporting that verification was incomplete due to OS-level privacy restrictions.

Q: Why use a client-side executable instead of a cloud-based agent?

A: The "one-shot" executable model respects the "Zero Trust" architecture of modern endpoints. It requires no permanent installation, leaves no footprint after execution, and avoids the "supply chain risk" of maintaining a persistent agent connection to a central server. It provides the consultant with immediate, point-in-time data without the liability of managing long-term access.

Q: Can this tool scale to enterprise-wide audits?

A: While designed for individual consultations, the core logic is modular. The JSON-structured data collection can be easily adapted to feed into a SIEM (Security Information and Event Management) system or a centralized dashboard if the delivery mechanism is shifted from a standalone executable to a network-login script.

7. Next Steps for Engagement

This tool represents a significant IP asset, codifying high-level security expertise into an automated, scalable product.

- **Review the Source:** The complete architectural implementation is available on GitHub: [Kajahkura/RSAT_Project](#)
- **Strategic Application:** This technology is ready for deployment in rapid-response incident handling, M&A technical due diligence, and standard remote workforce compliance auditing.

8. Conclusion

The development of the Remote Security Audit Tool (RSAT) demonstrates that complex operational challenges can be solved by combining the flexibility of Python with the authority of native system administration commands. By abstracting the deep architectural differences between Windows and macOS and handling privilege escalation autonomously, the tool bridges the gap between development scripts and production-grade software.

Ultimately, this architecture prioritizes robustness and user experience. It ensures that the critical "pass/fail" data required for security consultations is gathered reliably, regardless of the client's technical environment, delivering high-value security insights with minimal operational overhead.